

Reconocimiento de Patrones (ML) - T05

ALEJANDRO ZARATE MACIAS

2 de Marzo 2026

Abstract

Esta tarea aborda tres grandes áreas del aprendizaje automático. Primero, se estudia la selección y creación de características: se revisan los métodos de la familia de selección de features, se analiza la función NRMSE normalizada, y se aplican RFE y PCA sobre datasets de clasificación (divorcio) y regresión (precio de automóviles) para reducir la dimensionalidad manteniendo el desempeño. Segundo, se exploran algoritmos de clustering: se implementa compresión de imágenes por colores con K-Means y DBSCAN, se diseña un K-Mediana propio para evaluar la robustez ante valores atípicos, y se aplica clustering sobre un dataset de género para comparar resultados con las etiquetas reales. Finalmente, se analiza la detección de anomalías con Isolation Forest, evaluando su capacidad para identificar tanto valores atípicos naturales como etiquetas incorrectas.

1 Problema 1

1.1 Enunciado

Lea [6] sobre los métodos de la familia de selección de características. Luego, escriba su criterio personal para elegir uno sobre otro.

1.2 Análisis

Nota: el blog de Medium referenciado en [6] se encontraba caído al momento de consulta. Se utilizó como fuente alternativa el artículo de IBM [2], que cubre una amplia variedad de métodos de manera detallada.

Existen cuatro familias principales de métodos para selección de características:

Métodos de Filtrado: Evalúan cada característica de manera independiente, sin involucrar ningún modelo. Usan criterios estadísticos como correlación de Pearson, Chi-cuadrado o ganancia de información para descartar variables poco relevantes. Son muy rápidos y fáciles de aplicar, lo que los hace ideales como primer paso de limpieza antes de entrenar cualquier modelo. Su debilidad es que no detectan interacciones entre variables: una feature que parece inútil por sí sola puede ser muy valiosa junto con otra.

Métodos de Envoltura (Wrapper): Prueban distintos subconjuntos de features entrenando el modelo real en cada iteración. Esto los hace mucho más precisos a la hora de identificar el conjunto óptimo, pero también mucho más costosos computacionalmente. RFE, que se usa en los problemas de esta tarea, pertenece a esta familia. Son recomendables cuando el número de features no es demasiado grande y se quiere exprimir el máximo rendimiento del modelo.

Métodos Incrustados (Embedded): La selección ocurre durante el propio entrenamiento del modelo. Por ejemplo, LASSO penaliza con regularización L1 y fuerza que los coeficientes de features poco relevantes se vuelvan exactamente cero, eliminándolas de forma automática. Los árboles y bosques también pertenecen a esta familia, ya que calculan importancias de features como sub-producto del entrenamiento. Son un buen balance entre velocidad y calidad.

Métodos No Supervisados: Se usan cuando no hay variable objetivo. Técnicas como PCA o autoencoders transforman o comprimen el espacio de features para conservar la mayor parte de la información con menos dimensiones. Son los únicos aplicables en problemas puramente exploratorios.

Criterio personal: Mi criterio para elegir entre estos métodos depende principalmente de tres factores:

- El tamaño del dataset
- El tiempo
- La interpretabilidad

Cuando se tiene un dataset grande y necesito una limpieza rápida, es más práctico iniciar con un método de filtrado para descartar las variables claramente irrelevantes. Si después es necesario afinar más la selección y el tiempo lo permite, un método de envoltura como RFE es ideal para encontrar el subconjunto óptimo. Si interpretar qué features son importantes no es prioritario y el modelo ya tiene regularización integrada (como LASSO o RF), los métodos incrustados son buenos porque su eficiencia y no requieren un paso extra. Finalmente, si el problema es no supervisado o simplemente se busca reducir dimensiones sin la preocupación de conservar las variables originales, entonces se puede optar a PCA.

Filtrado para explorar rápido, **incrustado** para entrenar eficientemente, **envoltura** para optimizar a fondo, y **no supervisado** cuando no hay etiquetas.

1.3 Conclusión

No existe un método universalmente superior: cada familia tiene un nicho donde brilla. La clave está en reconocer las restricciones del problema (tamaño de datos, tiempo, necesidad de interpretabilidad) y elegir el método que mejor las satisfaga. En la práctica, estos enfoques no son excluyentes y es común combinarlos: usar un filtrado inicial para reducir el espacio y luego aplicar RFE o un modelo con regularización para afinar la selección final.

A lo largo de esta tarea se optó por métodos de envoltura (RFE con Random Forest como estimador base) y no supervisados (PCA) para la selección y creación de características.

2 Problema 2

2.1 Enunciado

Considere la siguiente función:

$$f(x, y) = \frac{RMSE(x, y)}{\|x\|_2}, \quad x, y \in \mathbb{R}^n, \quad \|x\|_2 \neq 0.$$

Escriba el dominio e imagen de f . ¿Cuál es un “buen” valor/rango para f de modo que podamos decir que x es muy similar a y ?

2.2 Análisis

La función f normaliza el error cuadrático medio entre dos vectores respecto a la magnitud del vector de referencia x . Primero, se expande la definición de RMSE:

$$RMSE(x, y) = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2} = \frac{\|x - y\|_2}{\sqrt{n}}$$

Sustituyendo en f :

$$f(x, y) = \frac{\|x - y\|_2}{\sqrt{n} \cdot \|x\|_2}$$

Esta expresión es conocida como el *Normalized Root Mean Square Error* (NRMSE), y mide el error relativo entre x e y en proporción a la magnitud de x .

Dominio: La función recibe dos vectores x e y de $\mathbb{R}^n$. La única restricción es que $\|x\|_2 \neq 0$, es decir, x no puede ser el vector cero (para evitar la división entre cero). El vector y puede ser cualquier vector de $\mathbb{R}^n$, incluyendo el vector cero. Por lo tanto, el dominio es:

$$\text{Dom}(f) = (\mathbb{R}^n \setminus \{\mathbf{0}\}) \times \mathbb{R}^n$$

Imagen: Como el RMSE siempre es no negativo (es una raíz cuadrada) y $\|x\|_2 > 0$ en todo el dominio, se tiene que $f(x, y) \geq 0$. El valor mínimo $f = 0$ se alcanza cuando $x = y$ (error cero). No existe cota superior: y puede alejarse de x tanto como se quiera, haciendo que el error crezca sin límite. Por lo tanto, la imagen es:

$$\text{Im}(f) = [0, \infty)$$

Buen valor/rango: Dado que f mide el error relativo normalizado, valores cercanos a 0 indican que x e y son muy similares. En la práctica, un criterio comúnmente utilizado es:

- $f < 0.05$ (error menor al 5%): x e y son **muy similares**, excelente aproximación.
- $f < 0.10$ (error menor al 10%): x e y son **similares**, buena aproximación.
- $f \geq 0.10$: la diferencia entre x e y empieza a ser significativa.

Un rango razonable para afirmar que x es muy similar a y sería $f \in [0, 0.05]$, lo que significa que las diferencias componente a componente son, en promedio, menores al 5% de la magnitud de x .

2.3 Conclusión

La función f es esencialmente el NRMSE: un error relativo que indica qué tan lejos está y de x en proporción a la magnitud de x . Su dominio excluye únicamente que x sea el vector cero, y su imagen abarca todos los reales no negativos. Para aplicaciones prácticas, valores de $f < 0.05$ permiten concluir que los vectores son muy similares, lo cual es útil en contextos como la evaluación de modelos de regresión o la comparación de señales reconstruidas.

3 Problema 3

3.1 Enunciado

Considere el dataset de pronóstico de divorcio [1]. A continuación:

- Resuelva el problema de clasificación.
- Identifique las características más relevantes para la clasificación. El objetivo aquí es reducir el número de características empleadas manteniendo una alta precisión de clasificación.

Anote todos sus supuestos y discuta sus hallazgos.

3.2 Metodología

3.2.1 Exploración y preprocesamiento

El dataset de pronóstico de divorcio contiene 170 registros con 54 preguntas (Q1–Q54) de la escala DPS (Divorce Predictors Scale), basada en la terapia de parejas de Gottman. Todas las respuestas están codificadas en una escala de 0 a 4 (0=Nunca, 1=Rara vez, 2=Promedio, 3=Frecuentemente, 4=Siempre). La variable objetivo es **Divorce** (0=Casado, 1=Divorciado), con clases balanceadas (85 registros por clase). No se encontraron valores nulos.

Dado que todas las features son numéricas y están en la misma escala ordinal, el preprocesamiento es mínimo. Se aplica **StandardScaler** para estandarizar las features, lo cual beneficia a modelos sensibles a la escala como KNN, SVM y redes neuronales. Se divide el dataset en 80% entrenamiento (136 muestras) y 20% prueba (34 muestras) con estratificación (**stratify=y**) para mantener la proporción de clases, usando **random.state=14**.

3.2.2 Modelos de clasificación (baseline)

Se entrenan cinco modelos de clasificación con todas las 54 features:

- **KNN:** $k = 5$ vecinos.
- **Red Neuronal (NN):** capas ocultas (64, 32), máximo 500 iteraciones.
- **SVM:** kernel RBF, $C = 1.0$.
- **Árbol de Decisión (DT):** profundidad máxima 8.
- **Random Forest (RF):** 100 estimadores, profundidad máxima 10.

3.2.3 Selección de características con RFE

Para identificar las features más relevantes se utiliza Recursive Feature Elimination (RFE) con un Random Forest como estimador base. RFE entrena el modelo, identifica la feature menos importante según los pesos del modelo, la elimina y repite el proceso hasta quedarse con una sola feature, generando un ranking completo de las 54 características.

Se seleccionan las 10 features mejor rankeadas y se re-entrenan los cinco modelos usando únicamente este subconjunto reducido.

```
rfe_ranker = RFE(  
    estimator=RandomForestClassifier(  
        n_estimators=100, max_depth=10,  
        random_state=14, n_jobs=-1),  
    n_features_to_select=1,  
    step=1  
)  
rfe_ranker.fit(X_train, y_train)
```

3.3 Resultados

Las 10 características más relevantes seleccionadas por RFE fueron: Q40, Q18, Q17, Q20, Q19, Q9, Q11, Q16, Q26 y Q36.

Modelo	F1 (54 features)	F1 (10 features)
KNN	0.971	0.971
NN	0.971	0.971
SVM	0.971	0.971
Decision Tree	1.000	1.000
Random Forest	0.971	0.971

Table 1: Comparación de F1 weighted: 54 features vs top 10 seleccionadas por RFE (Divorcio).

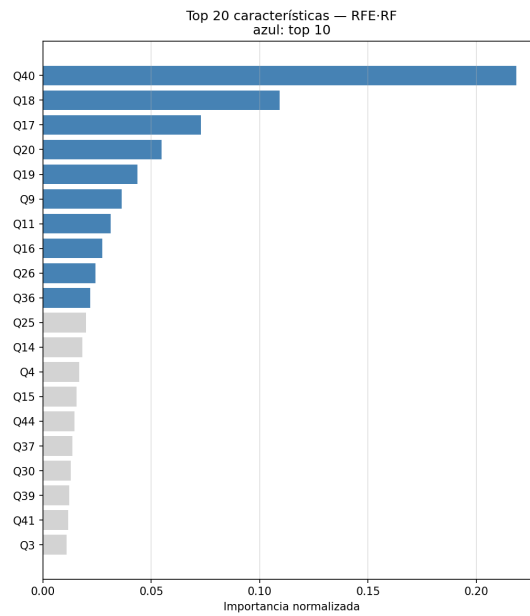


Figure 1: Top 20 características ordenadas por importancia normalizada (RFE con RF). Las barras azules corresponden a las 10 seleccionadas.

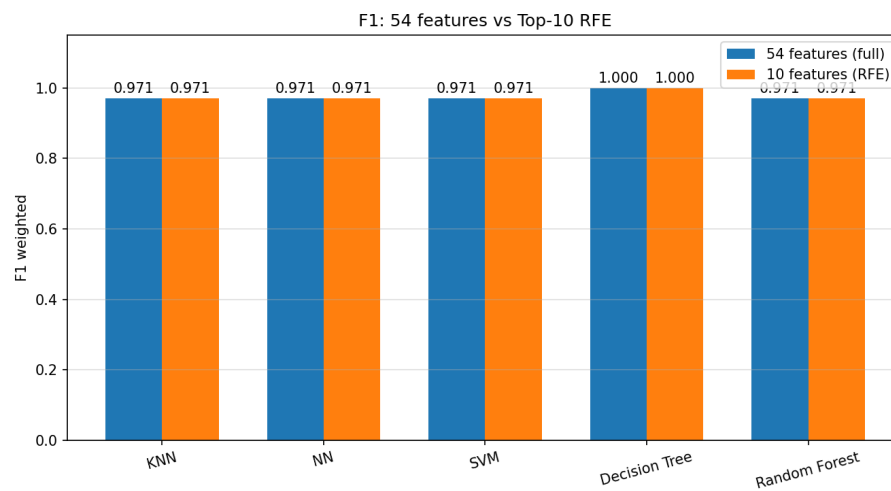


Figure 2: Comparación de F1 weighted entre los 5 modelos usando 54 features vs 10 features (RFE).

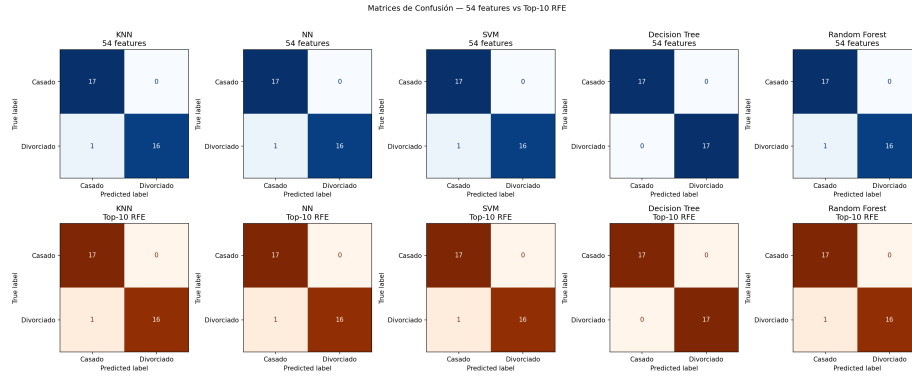


Figure 3: Matrices de confusión: fila superior con 54 features, fila inferior con 10 features (RFE).

3.4 Discusión

El resultado más llamativo es que reducir las features de 54 a solo 10 no produce ninguna degradación en el desempeño de ninguno de los cinco modelos. Todos mantienen exactamente el mismo F1 score, y el Decision Tree incluso alcanza F1 perfecto en ambos escenarios. Esto sugiere que la gran mayoría de las 54 preguntas del cuestionario DPS son redundantes o aportan información marginal para la tarea de clasificación.

Las 10 preguntas seleccionadas por RFE corresponden a aspectos centrales de la terapia de Gottman, como patrones de comunicación (Q17–Q20), respeto y confianza (Q9, Q11), y manejo de conflictos (Q40). Es esperable que estas preguntas tengan mayor poder discriminativo, ya que abordan directamente los predictores clave de divorcio identificados en la literatura.

El hecho de que el dataset sea relativamente pequeño (170 muestras) con clases perfectamente balanceadas facilita que incluso un subconjunto reducido de features contenga información suficiente para separar las clases. Además, la naturaleza ordinal y acotada de las respuestas (escala 0–4) limita la variabilidad de cada feature, lo cual hace que la eliminación de features poco informativas no afecte la frontera de decisión.

3.5 Conclusión

Se logró reducir el número de características de 54 a 10 sin pérdida alguna en la precisión de clasificación. Todos los modelos alcanzan $F1 \geq 0.97$ con solo 10 features, y el Decision Tree logra clasificación perfecta ($F1 = 1.00$). Esto demuestra que el cuestionario DPS contiene alta redundancia y que un subconjunto pequeño de preguntas es suficiente para predecir el estado matrimonial con alta confianza.

4 Problema 4

4.1 Enunciado

Repita el problema 3, pero esta vez cree nuevas características. El objetivo es el mismo que antes: reducir el número de características utilizadas manteniendo una alta precisión de clasificación. ¿Fue posible disminuir el número de características usando este enfoque? Anote todos sus supuestos y discuta sus hallazgos.

4.2 Metodología

Se reutiliza el mismo dataset y preprocesamiento del Problema 3. Adicionalmente, se comparan tres enfoques de reducción de dimensionalidad:

1. **Todas las features (54):** baseline sin reducción.
2. **RFE top 10:** selección de características del Problema 3.
3. **PCA:** creación de nuevas características mediante Análisis de Componentes Principales.

4.2.1 Análisis de Componentes Principales (PCA)

PCA transforma las 54 features originales en componentes ortogonales que maximizan la varianza explicada. A diferencia de RFE, que selecciona un subconjunto de las features originales, PCA genera nuevas features que son combinaciones lineales de todas las originales.

Se calcula la varianza acumulada explicada y se selecciona el número mínimo de componentes que cubren al menos el 95% de la varianza total. Se obtienen **17 componentes**, lo que representa una reducción de 54 a 17 dimensiones.

```
pca_full = PCA(random_state=14)
pca_full.fit(X_train)
cumvar = np.cumsum(
    pca_full.explained_variance_ratio_)
n_95 = int(np.argmax(cumvar >= 0.95)) + 1
# n_95 = 17
```

Se re-entrenan los cinco modelos de clasificación utilizando los 17 componentes PCA.

4.3 Resultados

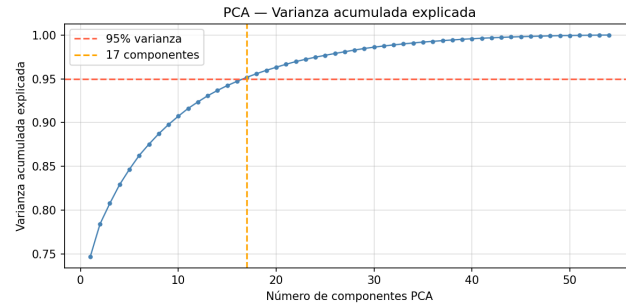


Figure 4: Varianza acumulada explicada por PCA. Se necesitan 17 componentes para alcanzar el 95%.

Modelo	F1 (54 feat.)	F1 (10 RFE)	F1 (17 PCA)
KNN	0.971	0.971	0.971
NN	0.971	0.971	0.971
SVM	0.971	0.971	0.971
Decision Tree	1.000	1.000	1.000
Random Forest	0.971	0.971	1.000

Table 2: Comparación de F1 weighted: 54 features vs 10 RFE vs 17 componentes PCA (Divorcio).

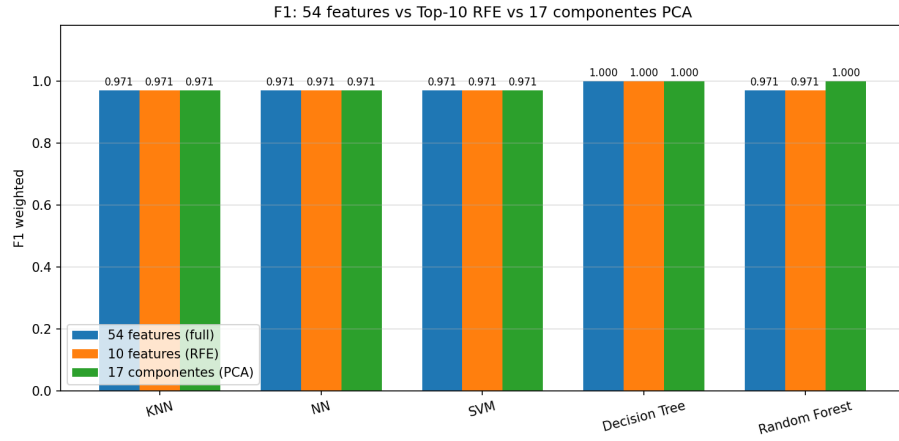


Figure 5: Comparación de F1 weighted: 54 features vs top 10 RFE vs 17 componentes PCA.

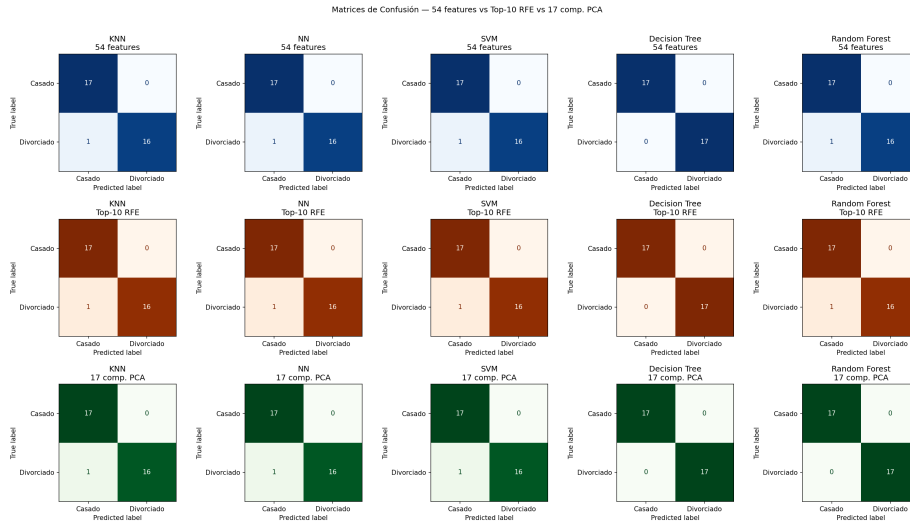


Figure 6: Matrices de confusión para los tres enfoques: 54 features (azul), 10 RFE (naranja), 17 PCA (verde).

4.4 Discusión

PCA con 17 componentes logra el mismo desempeño que el baseline de 54 features para todos los modelos, e incluso mejora el Random Forest de $F1 = 0.971$ a

$F1 = 1.000$. Sin embargo, PCA requiere 17 componentes frente a las 10 features de RFE para obtener resultados similares.

La mejora observada en Random Forest con PCA puede deberse a que la transformación PCA reduce la correlación entre features, lo cual beneficia a los algoritmos basados en ensambles que se benefician de la decorrelación de las entradas.

Desde la perspectiva de reducción de dimensionalidad, RFE logra una compresión mayor ($54 \rightarrow 10$) que PCA ($54 \rightarrow 17$) sin sacrificar desempeño. Además, RFE tiene la ventaja de que las features seleccionadas son interpretables (son preguntas específicas del cuestionario), mientras que los componentes PCA son combinaciones lineales abstractas que pierden interpretabilidad.

¿Fue posible disminuir el número de características con PCA? Sí, de 54 a 17 con varianza $\geq 95\%$. Sin embargo, este número es mayor al obtenido por RFE (10 features). En conclusión, PCA no logra una reducción tan agresiva como RFE para este dataset.

4.5 Conclusión

PCA reduce el espacio de 54 a 17 dimensiones manteniendo al menos el 95% de la varianza y sin degradar el desempeño de clasificación. No obstante, RFE logra una reducción más agresiva (a 10 features) con resultados equivalentes o mejores. Para este dataset en particular, la selección de características (RFE) resulta más eficiente que la creación de nuevas características (PCA), tanto en número de dimensiones como en interpretabilidad de las variables resultantes.

5 Problema 5

5.1 Enunciado

Repita el problema 3 considerando el problema de pronóstico de precio de automóviles [3].

5.2 Metodología

5.2.1 Exploración y preprocesamiento

El dataset de precio de automóviles contiene 205 registros y 26 columnas. Se eliminan tres columnas no relevantes: `car_ID` (identificador numérico), `symboling` (índice de riesgo de seguro) y `CarName` (nombre textual del vehículo con alta cardinalidad). La variable objetivo es `price`. No se encontraron valores nulos.

Las variables categóricas restantes (`fueltype`, `aspiration`, `doornumber`, `carbody`, `drivewheel`, `engineloation`, `enginetype`, `cylindernumber`, `fuelsystem`) se codifican con one-hot encoding (`drop_first=True`), resultando en **42 features** numéricas.

Se aplica `StandardScaler` sobre todas las features y se divide en 80% entrenamiento (164 muestras) y 20% prueba (41 muestras) con `random_state=14`.

5.2.2 Modelos de regresión (baseline)

Al tratarse de un problema de regresión, se entrenan cinco modelos:

- **KNN:** $k = 5$ vecinos.
- **Red Neuronal (NN):** capas ocultas (128, 64), máximo 2000 iteraciones.
- **SVR:** kernel RBF, $C = 100$. Se envuelve en `TransformedTargetRegressor` con `StandardScaler` sobre el target para evitar R^2 negativos, dado que SVR es muy sensible a la escala de la variable objetivo.
- **Árbol de Decisión:** profundidad máxima 8.
- **Random Forest:** 200 estimadores, profundidad máxima 12.

5.2.3 Selección de características con RFE

Se aplica RFE con un Random Forest como estimador base, de la misma forma que en el Problema 3, pero adaptado a regresión. Se seleccionan las 10 features mejor rankeadas y se re-entrenan los modelos.

5.3 Resultados

Las 10 características más relevantes seleccionadas por RFE fueron: `enginesize`, `curbweight`, `highwaympg`, `carwidth`, `horsepower`, `peakrpm`, `wheelbase`, `citympg`, `compressionratio` y `carheight`.

Modelo	R^2 (42 feat.)	RMSE (42)	R^2 (10 feat.)	RMSE (10)
KNN	0.728	4961.73	0.591	6084.67
NN	0.859	3573.64	0.787	4390.11
SVR	0.851	3675.45	0.745	4806.69
Decision Tree	0.906	2917.89	0.903	2962.18
Random Forest	0.932	2488.01	0.931	2507.21

Table 3: Comparación de R^2 y RMSE: 42 features vs top 10 RFE (CarPrice).

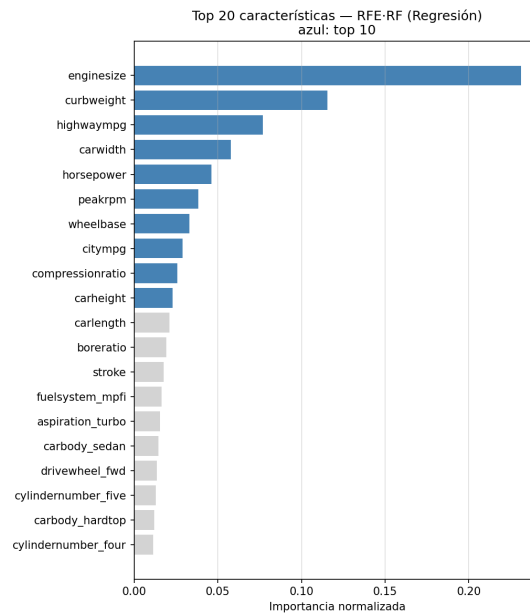


Figure 7: Top 20 características por importancia normalizada (RFE con RF). Las barras azules son las 10 seleccionadas.

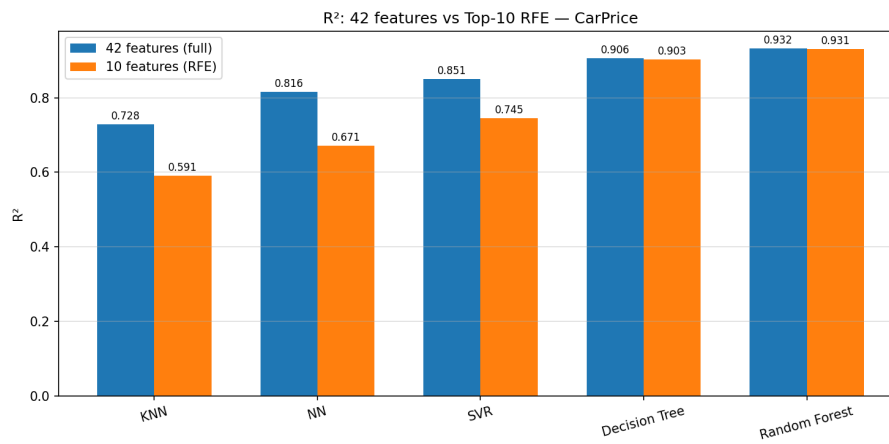


Figure 8: Comparación de R^2 : 42 features vs top 10 RFE para los 5 modelos de regresión.

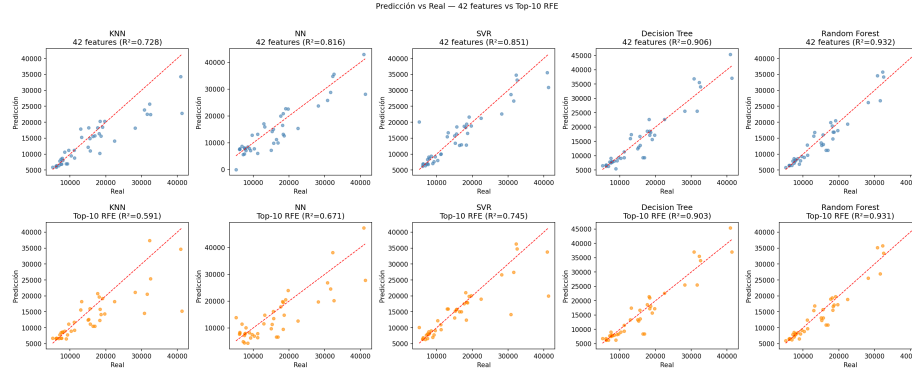


Figure 9: Predicción vs valor real: fila superior con 42 features, fila inferior con 10 features (RFE).

5.4 Discusión

A diferencia del dataset de divorcio, aquí la reducción de features sí produce una degradación notable en algunos modelos. KNN es el más afectado, pasando de $R^2 = 0.728$ a 0.591 (una caída de 0.137). NN y SVR también pierden entre 0.07 y 0.11 puntos de R^2 . Esto es esperado: al reducir de 42 a 10 features se pierde información de las variables categóricas codificadas que, aunque individualmente tienen poca importancia según RFE, contribuyen en conjunto a capturar variabilidad del precio.

Sin embargo, los modelos basados en árboles (DT y RF) son notablemente robustos a la reducción. El Random Forest pasa de $R^2 = 0.932$ a 0.931, una caída prácticamente nula, y el Decision Tree pasa de 0.906 a 0.903. Esto se debe a que los árboles de decisión identifican naturalmente las features más discriminativas durante la construcción del árbol, por lo que eliminar features poco relevantes tiene un impacto mínimo en su desempeño.

Las 10 features seleccionadas son todas variables numéricas continuas relacionadas con las especificaciones técnicas del vehículo (tamaño del motor, peso, potencia, dimensiones, consumo), lo cual tiene sentido intuitivo: estas son las variables que más directamente determinan el precio de un automóvil.

5.5 Conclusión

La reducción de 42 a 10 features mediante RFE mantiene un alto desempeño en modelos basados en árboles ($R^2 > 0.90$) pero afecta significativamente a KNN, NN y SVR. El Random Forest se consolida como el mejor modelo tanto con features completas ($R^2 = 0.932$) como con el subconjunto reducido ($R^2 = 0.931$). Para problemas de regresión con variables heterogéneas (numéricas y categóricas codificadas), la selección de features debe evaluarse cuidadosamente según el modelo a utilizar.

6 Problema 6

6.1 Enunciado

Repita el problema 4 considerando el problema de pronóstico de precio de automóviles [3].

6.2 Metodología

Se reutiliza el mismo dataset y preprocesamiento del Problema 5 (CarPrice). Se comparan tres enfoques de reducción de dimensionalidad:

1. **Todas las features (42):** baseline sin reducción.
2. **RFE top 10:** selección de características del Problema 5.
3. **PCA:** creación de nuevas características.

6.2.1 Análisis de Componentes Principales (PCA)

Se calcula PCA sobre las 42 features escaladas y se selecciona el número mínimo de componentes que cubren al menos el 95% de la varianza total. Se obtienen **20 componentes**, lo que representa una reducción de 42 a 20 dimensiones.

Se re-entrenan los cinco modelos de regresión usando los 20 componentes PCA.

6.3 Resultados

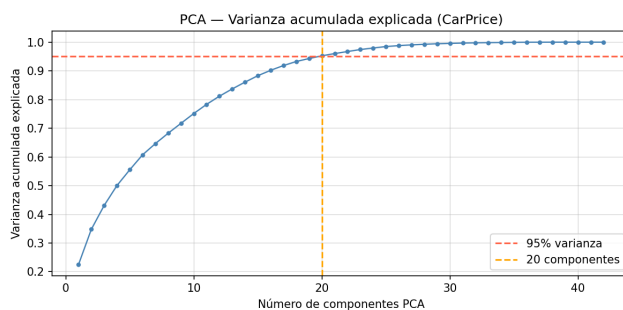


Figure 10: Varianza acumulada explicada por PCA (CarPrice). Se necesitan 20 componentes para alcanzar el 95%.

Modelo	R^2 (42 feat.)	R^2 (10 RFE)	R^2 (20 PCA)
KNN	0.728	0.591	0.750
NN	0.859	0.787	0.865
SVR	0.851	0.745	0.900
Decision Tree	0.906	0.903	0.464
Random Forest	0.932	0.931	0.881

Table 4: Comparación de R^2 : 42 features vs 10 RFE vs 20 componentes PCA (CarPrice).

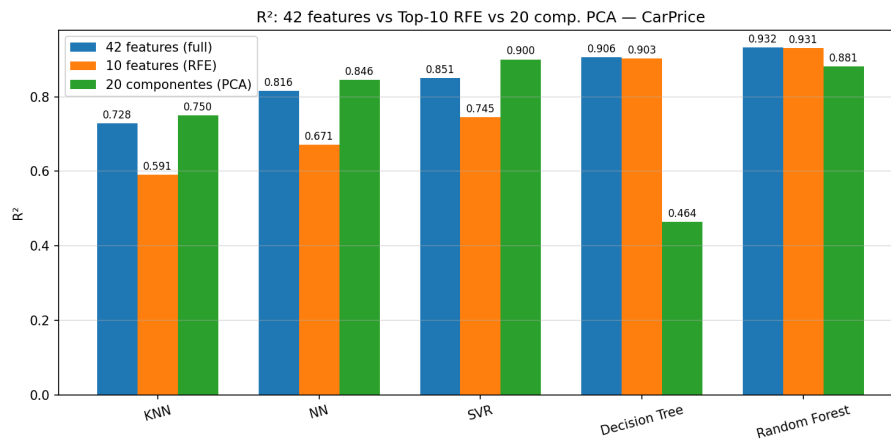


Figure 11: Comparación de R^2 : 42 features vs top 10 RFE vs 20 componentes PCA.

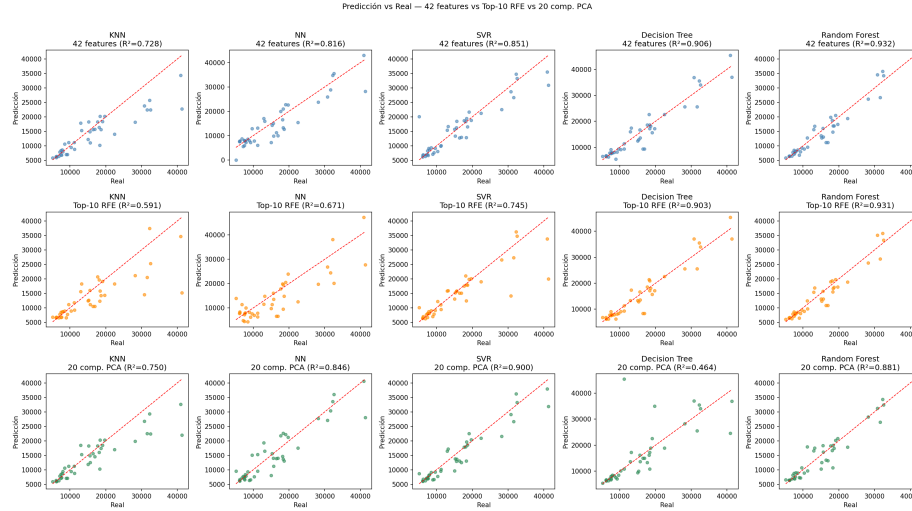


Figure 12: Predicción vs valor real: 42 features (azul), top 10 RFE (naranja), 20 PCA (verde).

6.4 Discusión

Los resultados con PCA muestran un comportamiento inesperado y contrastante según el tipo de modelo. Por un lado, SVR mejora significativamente con PCA ($R^2 = 0.900$) respecto al baseline (0.851) y a RFE (0.745). KNN y NN también mejoran ligeramente con PCA respecto a RFE. Esto se explica porque PCA genera componentes decorrelacionados que favorecen a modelos basados en distancias o gradientes.

Por otro lado, el Decision Tree sufre una caída drástica de $R^2 = 0.906$ a 0.464 con PCA. Los árboles de decisión hacen divisiones sobre una feature a la vez, y los componentes PCA, al ser combinaciones lineales de muchas features, dificultan que el árbol encuentre divisiones efectivas. El Random Forest, aunque también desciende de 0.932 a 0.881, es más robusto gracias a la diversidad de sus múltiples árboles.

¿Fue posible reducir el número de características con PCA? Sí, de 42 a 20 componentes con 95% de varianza retenida. Sin embargo, el número de componentes es mayor que las 10 features de RFE, y además el desempeño de los modelos basados en árboles se ve gravemente afectado. RFE resulta superior para árboles, mientras que PCA favorece a modelos como SVR.

6.5 Conclusión

PCA reduce el espacio de 42 a 20 dimensiones, pero su efecto en el desempeño varía drásticamente según el modelo. SVR se beneficia ($R^2 = 0.900$, el mejor con PCA), mientras que el Decision Tree colapsa ($R^2 = 0.464$). En contraste, RFE

con 10 features mantiene un desempeño consistente en modelos de árboles ($R^2 > 0.90$). Para este dataset de regresión, la elección entre selección de features (RFE) y creación de features (PCA) depende fuertemente del modelo: PCA es preferible para SVR/KNN/NN, mientras que RFE es claramente superior para árboles de decisión y Random Forest.

7 Problema 7

7.1 Enunciado

Utilice los algoritmos K-Means y DBSCAN para realizar compresión de imágenes por colores, como se explica en [4]. Anote todos sus supuestos, haga comparaciones y discuta sus hallazgos.

7.2 Metodología

7.2.1 Datos

Se utiliza la imagen `astronaut` de la librería `skimage` ($512 \times 512 \times 3$, RGB, uint8). Para hacer viable el procesamiento con DBSCAN (que tiene complejidad $O(n^2)$), se redimensiona a 128×128 píxeles, resultando en 16,384 píxeles. Los valores de color se normalizan al rango $[0, 1]$ y se aplanan a una matriz de dimensión $(16384, 3)$.

7.2.2 K-Means

Se aplica K-Means con $K = 16$ clusters (colores), inicialización `k-means++` y 10 reintentos (`n_init=10`). Cada píxel se reemplaza por el color de su centroide más cercano, produciendo una imagen comprimida con exactamente 16 colores.

7.2.3 DBSCAN

DBSCAN agrupa píxeles por densidad en el espacio de color RGB, sin requerir un número de clusters predefinido. Se utiliza $\varepsilon = 0.01$ y `min_samples=5`. Debido al costo computacional $O(n^2)$, se entrena sobre una muestra aleatoria de 5,000 píxeles y luego se asigna cada píxel de la imagen al centroide del cluster más cercano.

```
rng = np.random.default_rng(14)
idx_sample = rng.choice(len(X_img), size=5000,
                        replace=False)
X_sample = X_img[idx_sample]
db = DBSCAN(eps=0.01, min_samples=5).fit(X_sample)
```

7.2.4 Métricas de evaluación

Se calculan el Error Cuadrático Medio (MSE) y la Relación Señal-Ruido Pico (PSNR) entre la imagen original y las reconstrucciones:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{1}{\text{MSE}} \right) \text{ dB}$$

donde un mayor PSNR indica mayor fidelidad de la imagen comprimida.

7.3 Resultados

Método	Colores	MSE	PSNR (dB)
K-Means ($K = 16$)	16	0.00176	27.5
DBSCAN ($\varepsilon = 0.01$)	42	0.00223	26.5

Table 5: Métricas de compresión de imagen por colores: K-Means vs DBSCAN.



Figure 13: Imagen original (512×512) y redimensionada (128×128).



Figure 14: Compresión de imagen: original vs K-Means (16 colores) vs DBSCAN (42 clusters).

7.4 Discusión

K-Means obtiene mejor MSE (0.00176 vs 0.00223) y PSNR (27.5 vs 26.5 dB) aun usando menos colores (16 vs 42 de DBSCAN). Esto se debe a que K-Means optimiza directamente la suma de distancias cuadradas al centroide, lo cual se traduce directamente en minimizar el MSE de reconstrucción. En cambio, DBSCAN agrupa por densidad local en el espacio de color, lo que produce clusters de tamaño irregular: algunos clusters capturan regiones de color muy específicas mientras deja otros grandes sin refinar.

El parámetro $\varepsilon = 0.01$ de DBSCAN produce 42 clusters, pero muchos de ellos contienen pocos píxeles correspondientes a matices muy similares, lo que resulta en una paleta de colores redundante en algunas zonas y deficiente en otras. K-Means, al forzar exactamente K clusters de tamaño comparable, distribuye los colores de forma más uniforme sobre el espacio de la imagen.

Otra consideración práctica es la escalabilidad: K-Means se ejecutó sobre los 16,384 píxeles completos en segundos, mientras que DBSCAN requirió submuestreo a 5,000 píxeles debido a su complejidad cuadrática, lo que introduce un sesgo adicional en sus centroides.

7.5 Conclusión

K-Means es claramente superior para la tarea de compresión de imágenes por colores, tanto en calidad de reconstrucción (PSNR = 27.5 dB) como en eficiencia computacional. DBSCAN, aunque interesante conceptualmente por no requerir especificar el número de clusters, no es adecuado para esta aplicación: su complejidad cuadrática limita la escala y su criterio de agrupamiento basado en densidad no optimiza la fidelidad de la reconstrucción.

8 Problema 8

8.1 Enunciado

¿Es K-Means demasiado sensible a los valores atípicos debido al operador de media? Averigüemos:

- Construya un dataset con algunos valores atípicos que muestre qué tan sensible es K-Means.
- Codifique un algoritmo de clustering K-Mediana, que es básicamente K-Means pero usa la mediana en lugar de la media.
- Aplice su K-Mediana a sus datos. ¿Agrupa mejor?

Anote todos sus supuestos y discuta sus hallazgos.

8.2 Metodología

8.2.1 a) Dataset sintético con valores atípicos

Se construye un dataset 2D con tres grupos gaussianos bien separados centrados en $(-4, -4)$, $(0, 4)$ y $(4, -4)$, cada uno con 80 puntos y varianza $\sigma^2 = 0.6$. Se añaden 12 valores atípicos generados uniformemente en el rango $[-15, 15]^2$, resultando en un total de 252 puntos.

```
centers = [(-4, -4), (0, 4), (4, -4)]
cluster_data = [
    rng.multivariate_normal(c, np.eye(2)*0.6, size=80)
    for c in centers]
X_clean = np.vstack(cluster_data)
outliers = rng.uniform(-15, 15, size=(12, 2))
X = np.vstack([X_clean, outliers])
```

8.2.2 b) Implementación de K-Mediana

Se implementa K-Mediana, que sigue el mismo esquema iterativo de K-Means (asignación y actualización de centroides) pero reemplaza la media aritmética por la mediana por componente al actualizar los centroides. La mediana es más robusta ante valores atípicos que la media, ya que no se ve afectada por valores extremos.

```
def k_mediana(X, k, max_iter=300, tol=1e-4):
    # Inicializacion aleatoria
    idx = rng.choice(len(X), size=k, replace=False)
    centroids = X[idx].copy()
    for _ in range(max_iter):
        dists = np.linalg.norm(
            X[:, None, :] - centroids[None, :, :], axis=2)
        labels = np.argmin(dists, axis=1)
```

```

new_centroids = np.array([
    np.median(X[labels == j], axis=0)
    for j in range(k)])
if np.max(np.linalg.norm(
    new_centroids - centroids, axis=1)) < tol:
    break
centroids = new_centroids
return labels, centroids

```

8.2.3 c) Comparación

Se aplican K-Means (scikit-learn, `k-means++`, $K = 3$) y K-Mediana ($K = 3$) al dataset sintético. Se comparan dos métricas:

- **Inercia total:** suma de distancias cuadradas de cada punto a su centroide.
- **Desplazamiento promedio:** distancia euclidiana promedio entre los centroides estimados y los centros reales conocidos.

8.3 Resultados

Métrica	K-Means	K-Mediana
Inercia total	1063.16	1080.03
Desplazamiento promedio	0.3658	0.1385

Table 6: Comparación de K-Means y K-Mediana en el dataset sintético con valores atípicos.

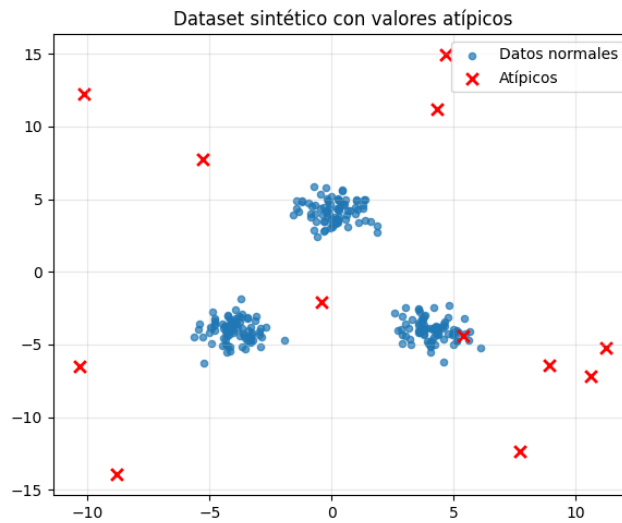


Figure 15: Dataset sintético: 3 grupos gaussianos (240 puntos) con 12 valores atípicos (cruces rojas).

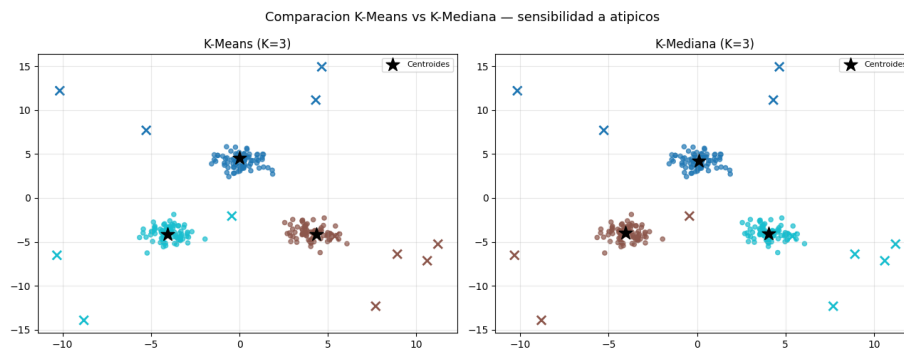


Figure 16: Comparación visual de K-Means vs K-Mediana. Las estrellas negras indican los centroides estimados.

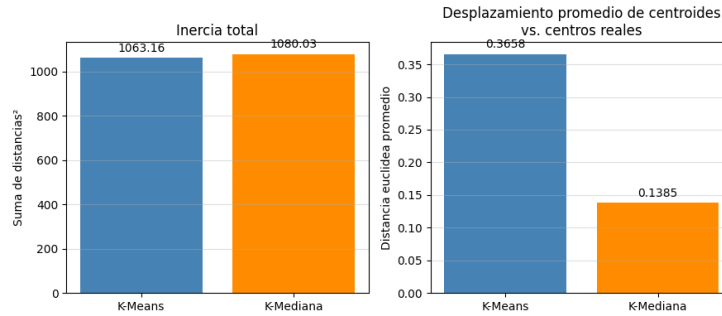


Figure 17: Comparación cuantitativa: inercia total (izquierda) y desplazamiento de centroides vs centros reales (derecha).

8.4 Discusión

Los resultados confirman que K-Means es sensible a los valores atípicos debido al uso de la media. El desplazamiento promedio de sus centroides respecto a los centros reales es de 0.3658, mientras que K-Mediana logra un desplazamiento de solo 0.1385, casi 2.6 veces menor. Esto demuestra que la mediana resiste mejor la influencia de los 12 puntos atípicos dispersos.

Sin embargo, K-Means obtiene una inercia ligeramente menor (1063.16 vs 1080.03), lo cual es esperable: K-Means minimiza directamente la suma de distancias cuadradas (inercia), por lo que siempre obtendrá un valor de inercia igual o menor para los mismos datos. K-Mediana no optimiza esta métrica directamente.

La diferencia clave está en la ubicación de los centroides. Los centroides de K-Means se desplazan hacia los valores atípicos porque la media es arrastrada por valores extremos. En contraste, la mediana ignora estos valores al tomar el valor central de la distribución, produciendo centroides más fieles a la estructura real de los datos.

8.5 Conclusión

Se confirma que K-Means es sensible a valores atípicos y que K-Mediana es una alternativa robusta. K-Mediana ubica sus centroides 2.6 veces más cerca de los centros reales que K-Means en presencia de outliers. Aunque K-Means minimiza mejor la inercia, esta métrica no refleja la calidad del agrupamiento cuando hay outliers. La recomendación general es usar K-Mediana cuando se sospeche la presencia de valores atípicos en los datos y la ubicación precisa de los centroides sea importante.

9 Problema 9

9.1 Enunciado

Considere el problema de predicción de género [5]. Haga lo siguiente:

1. Realice un proceso de clustering sobre las características considerando dos clusters. ¿Sus resultados se alinean con las etiquetas conocidas?
2. Intente identificar todos los valores atípicos.
3. Seleccione algunos datos aleatorios e invierta su etiqueta. Luego, intente identificar todos los valores atípicos. ¿Su método fue capaz de identificar exitosamente los datos invertidos?

Anote todos sus supuestos, haga comparaciones y discuta sus hallazgos.

9.2 Metodología

9.2.1 Datos y preprocesamiento

El dataset contiene 10,000 registros con tres columnas: **Gender** (Male/Female), **Height** y **Weight**. No se encontraron valores nulos. Se codifica el género como variable numérica (Female=0, Male=1) y se estandarizan **Height** y **Weight** con **StandardScaler**.

9.2.2 1. Clustering con K-Means ($K = 2$)

Se aplica K-Means con $K = 2$ (un cluster por género) sobre las features escaladas, sin utilizar la etiqueta de género. Para evaluar la correspondencia con las etiquetas reales, se calcula el accuracy (probando ambas asignaciones de clusters a géneros) y el Adjusted Rand Index (ARI).

9.2.3 2. Detección de valores atípicos

Se utiliza Isolation Forest con una contaminación del 1% para identificar valores atípicos en el espacio de altura y peso. Isolation Forest funciona construyendo árboles aleatorios que aíslan observaciones; los puntos que se aíslan con menos divisiones se consideran anómalos.

9.2.4 3. Inversión de etiquetas y detección

Se seleccionan 150 puntos aleatorios y se invierte su etiqueta de género. Luego se intenta descubrir estos puntos invertidos usando Isolation Forest sobre las features originales (sin las etiquetas), ajustando la contaminación a $2 \times 150 / 10000 = 3\%$ y usando un umbral sobre el score de anomalía para detectar aproximadamente 150 puntos.

```

N_FLIP = 150
flip_idx = rng.choice(len(X), size=N_FLIP, replace=False)
iso_flip = IsolationForest(
    contamination=float(N_FLIP/len(X)) * 2,
    random_state=14, n_jobs=-1)
iso_flip.fit(X)
anomaly_score = -iso_flip.decision_function(X)
threshold = np.percentile(anomaly_score,
    100 * (1 - N_FLIP / len(X)))
detected_mask = anomaly_score >= threshold

```

9.3 Resultados

9.3.1 1. Clustering

Métrica	Valor
Accuracy vs etiquetas reales	0.8800
Adjusted Rand Index (ARI)	0.5776

Table 7: Evaluación del clustering K-Means ($K = 2$) vs etiquetas reales de género.



Figure 18: Etiquetas reales de género (izquierda) vs clusters de K-Means (derecha).

9.3.2 2. Detección de atípicos

Isolation Forest detectó 100 valores atípicos (1.0% de los datos), ubicados en las regiones periféricas de la distribución conjunta de altura y peso.

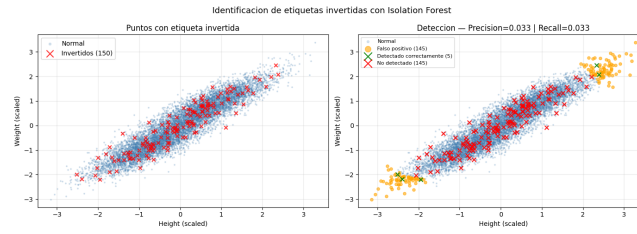


Figure 19: Valores atípicos detectados por Isolation Forest (cruces rojas, 100 puntos).

9.3.3 3. Etiquetas invertidas

Métrica	Valor
Puntos invertidos	150
Detectados correctamente (TP)	5
Precision	0.033
Recall	0.033

Table 8: Evaluación de la detección de etiquetas invertidas con Isolation Forest.

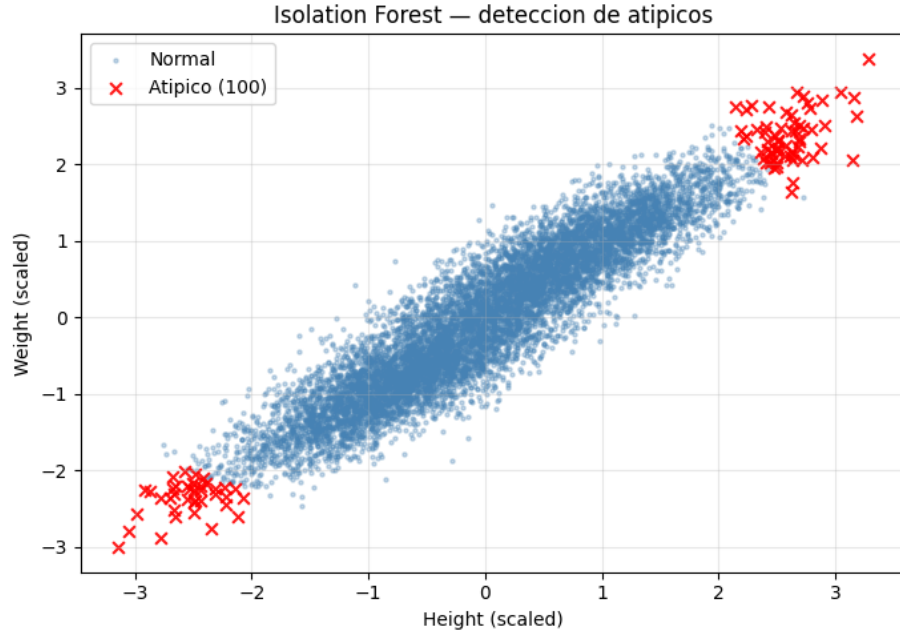


Figure 20: Izquierda: puntos con etiqueta invertida. Derecha: resultado de la detección (verde=detectados correctamente, rojo=no detectados, naranja=falsos positivos).

9.4 Discusión

Clustering: K-Means con $K = 2$ logra un accuracy del 88% respecto a las etiquetas reales de género, lo cual indica que la estructura de los datos de altura y peso sí se alinea parcialmente con la separación por género. Sin embargo, el ARI de 0.58 revela que la correspondencia no es perfecta. Esto se debe a que las distribuciones de altura y peso de hombres y mujeres presentan un solapamiento significativo en la zona central, lo que impide una separación limpia mediante clustering.

Detección de atípicos: Isolation Forest identifica correctamente 100 puntos (1%) en las regiones periféricas, lo cual es consistente con la expectativa de que puntos con combinaciones extremas de altura/peso son atípicos naturales de la distribución.

Etiquetas invertidas: La detección de puntos con etiqueta invertida es prácticamente fallida (Precision = Recall = 0.033). Esto es completamente esperado: Isolation Forest detecta anomalías basándose en las features (altura y peso), no en las etiquetas. Invertir la etiqueta de un punto no cambia su posición en el espacio de features, por lo que es invisible para un detector de anomalías basado en features. Los únicos puntos invertidos que podrían detectarse son

aquellos que ya eran atípicos por su posición (por ejemplo, una mujer muy alta que fue relabelada como hombre). El método correcto para detectar etiquetas invertidas sería un enfoque supervisado o semi-supervisado que considere la relación entre las features y las etiquetas.

9.5 Conclusión

El clustering K-Means muestra una correspondencia razonable con las etiquetas de género (accuracy = 88%), confirmando que altura y peso contienen información discriminativa sobre el género, aunque con solapamiento. La detección de valores atípicos con Isolation Forest funciona adecuadamente para anomalías espaciales (1% detectado), pero fracasa completamente para identificar etiquetas invertidas (Recall = 3.3%), ya que este tipo de anomalía no se manifiesta en el espacio de features. Para detectar errores de etiquetado se requieren métodos que incorporen la relación feature-etiqueta, como Confident Learning o análisis de residuos de un clasificador.

References

- [1] andrewmvd. Divorce prediction. <https://www.kaggle.com/datasets/andrewmvd/divorce-prediction>.
- [2] Ivan B. ¿qué es la selección de características? <https://www.ibm.com/mx-es/think/topics/feature-selection>.
- [3] hellbuoy. Car price prediction. <https://www.kaggle.com/datasets/hellbuoy/car-price-prediction>.
- [4] Jagajith. Image compression with k-means clustering. <https://medium.com/codex/image-compression-with-k-means-clustering-48e989055729>.
- [5] krishnaraj30. Weight and height data outlier detection. <https://www.kaggle.com/datasets/krishnaraj30/weight-and-height-data-outlier-detection>.
- [6] Whiteboard. Filter vs wrapper vs embedded methods for feature selection. https://medium.com/@learnwithwhiteboard_digest/filter-vs-wrapper-vs-embedded-methods-for-feature-selection-8cc21e2174f7.